Entropy Pooling: A Comprehensive Report

Introduction to Entropy Pooling Entropy Pooling (EP), originally introduced by Attilio Meucci (2008), is a robust and flexible method for incorporating subjective views or stress tests into a prior distribution of financial scenarios. EP aims to compute a posterior probability distribution over simulated scenarios that is as close as possible to the prior, while still satisfying the users specified views. This is done by minimizing the Kullback-Leibler divergence (relative entropy) between the posterior and the prior distributions, under a set of linear constraints encoding the views.

Mathematical Formulation:
Let: R be a (S*I) matrix of simulated scenarios.
p belongs to (S*1) be the vector of prior scenario probabilities.
q belongs to (S*1) be the vector of posterior scenario probabilities.

Objective: Minimize the relative entropy: q = arg min x(transpose) (ln x - ln p)
Subject to: Ax = b (equality constraints)
Gx <= h (inequality constraints)
Summation xi = 1 and xi >=0 (probability simplex)

Code Implementation Summary

Basic Entropy Pooling (EP) Implementation

```
q = ft.entropy_pooling(p, A, b)
relative_entropy = q.T @ (np.log(q) - np.log(p))
effective_number_scenarios = np.exp(-relative_entropy)
print(f'RE = {np.round(relative_entropy[0, 0], 3)}.')
print(f'Relative ENS = {np.round(effective_number_scenarios[0, 0], 3)}.')
```

```
RE = 0.047.
Relative ENS = 0.954.
```

Various Parameters

```
R = R_df.values
S = R.shape[0]
p = np.ones((S, 1)) / S
means_prior = p.T @ R
vols_prior = np.sqrt(p.T @ (R - means_prior)**2)

mean_rows = R[:, 2:7].T
vol_rows = (R[:, 2:6] - means_prior[:, 2:6]).T**2
skew_row = ((R[:, 4] - means_prior[:, 4]) / vols_prior[:, 4])**3
kurt_row = ((R[:, 4] - means_prior[:, 4]) / vols_prior[:, 4])**4
corr_row = (R[:, 2] - means_prior[:, 2]) * (R[:, 3] - means_prior[:, 3])

A = np.vstack((np.ones((1, S)), mean_rows, vol_rows[0:-1, :], corr_row[np.newaxis, :]))
b = np.vstack(([1], means_prior[:, 2:6].T, [0.1], vols_prior[:, 2:5].T**2,
               [0.5 * vols_prior[0, 2] * vols_prior[0, 3]]))
G = np.vstack((vol_rows[-1, :], skew_row, -kurt_row))
h = np.array([[0.2**2], [-0.75], [-3.5]])
```

Sequential Heuristics(H1)

To overcome limitations of Entropy Pooling, Sequential Entropy Pooling (SeqEP) was proposed. It repeatedly applies the EP algorithm in stages, refining intermediate distributions and resolving nonlinearities by progressively fixing only the values that were updated in earlier steps.

Overview of Algorithm and code:

1. We categorise views into classes C0, C1, ….Ci as follows:
    a. C0: Means
    b. C1: Variances (requires mean)
    c. C2: Skewness (requires mean, variance)
    d. C3: Correlations (requires two means, two variances)
    e. C^{-}: Parameters directly expressible linearly in posterior probabilities
2. We apply EP using views in C0 and prior probability(p) and extract posterior probabilities(q0).
3. Then, sequentially apply this algorithm on all views.

```
# C0 views
mean_row = R[:, 1][np.newaxis]
A0 = np.vstack((np.ones((1, S)), mean_row))
b0 = np.array([[1.], [0.1]])
q0 = ft.entropy_pooling(p, A0, b0)
```

```
ft.simulation_moments(R_df, q0)
```

	Mean	Volatility	Skewness	Kurtosis
Gov & MBS	0.052059	0.035737	-0.266208	2.265610
Corp IG	0.100000	0.044309	-0.181936	2.128673
Corp HY	0.096963	0.054581	-0.187032	3.268894
EM Debt	0.163537	0.080470	-0.513780	2.418053
DM Equity	0.138907	0.143354	-0.298493	2.541248
EM Equity	0.236400	0.252120	0.413821	4.009514
Private Equity	0.309116	0.298377	-0.188268	2.031546
Infrastructure	0.173938	0.155932	-0.087321	1.786859
Real Estate	0.096609	0.083638	-0.573675	2.526741
Hedge Funds	0.126293	0.074791	-0.099078	4.009661

```
# C1 views
G = np.vstack((vol_rows[-1, :], skew_row, -kurt_row))
h = np.array([[0.2**2], [-0.75], [-3.5]])
means0 = q0.T @ R
G1 = np.vstack((np.ones((1, S)),vol_rows[-1, :]))
h1 = np.array([[1.],[0.2**2]])
q1 = ft.entropy_pooling(p, A0, b0,G1,h1)
```

```
ft.simulation_moments(R_df, q1)
```

	Mean	Volatility	Skewness	Kurtosis
Gov & MBS	0.054774	0.035654	-0.371903	2.323417
Corp IG	0.100000	0.044837	-0.170081	2.082054
Corp HY	0.088799	0.049755	-0.112197	3.493298
EM Debt	0.155484	0.078428	-0.382654	2.199343
DM Equity	0.125618	0.136872	-0.336705	2.008845
EM Equity	0.168497	0.179375	-0.297939	2.611375
Private Equity	0.285589	0.293248	-0.161561	1.781404
Infrastructure	0.173096	0.157480	-0.074595	1.765372
Real Estate	0.089374	0.084538	-0.400003	2.281638
Hedge Funds	0.115472	0.066862	-0.501841	2.197843

4. Following this, we calculate RE and ENS and plot the results.

```
relative_entropy1 = q3.T @ (np.log(q3) - np.log(p))
effective_number_scenarios1 = np.exp(-relative_entropy1)
print(f'RE = {np.round(relative_entropy1[0, 0], 3)}.')
print(f'Relative ENS = {np.round(effective_number_scenarios1[0, 0], 3)}.')
```

```
RE = 4.391.
Relative ENS = 0.012.
```

H2 Heuristic of Sequential Entropy Pooling

Introduction

The **H2 heuristic** is a practical approach within the Sequential Entropy Pooling (SEP) framework for incorporating investor views into a prior distribution in a step-by-step manner. In contrast to traditional Entropy Pooling, which incorporates all views simultaneously, SEP breaks the process into sequential stages—typically ordered from less to more complex views (e.g., mean → variance → skewness → kurtosis).

In the **H2 heuristic**, the **posterior distribution from one step becomes the prior for the next step**. This contrasts with the H1 heuristic, which always uses the original prior p at each step, ignoring the updates made in earlier rounds.

Conceptual Overview of H2 Heuristic

The H2 heuristic in Sequential Entropy Pooling (SEP) is built on the intuitive idea that investor views often exhibit a natural order of dependency. For example:

- Volatility views typically depend on a fixed mean,

- Skewness views depend on both mean and variance,
- Kurtosis depends on all lower moments.

Incorporating such interdependent views **simultaneously** through traditional Entropy Pooling (EP) can often lead to **logical inconsistencies or infeasible optimization problems**.
The H2 approach solves this by **updating the distribution sequentially**, one class of views at a time—ensuring that dependencies are respected and computational feasibility is maintained.

Here is the algorithm for H2 heuristics

Algorithm 2 (H2)

for $i \in \{0, 1, ..., I\}$
  if $\mathcal{V}_i \neq \emptyset$, compute $q_i = EP\left(\mathcal{V}^i, \theta_{i-1}, q_{i-1}\right)$ and $\theta_i = f\left(R, q_i\right)$
  else $q_i = q_{i-1}$ and $\theta_i = \theta_{i-1}$
if $\bar{\mathcal{V}} \neq \emptyset$, compute $q = EP\left(\mathcal{V}, \theta_I, q_I\right)$
else $q = q_I$
return q

Implementation

We now explain the H2 procedure through the actual code implementation.

Step1 : Incorporating mean views(C0 class)
In this step, we update the **prior distribution p** to reflect a **mean view on a specific asset (index 1)**, while ensuring the total probability sums to 1.

```
# C0 views
mean_row = R[:, 1][np.newaxis]
A0 = np.vstack((np.ones((1, S)), mean_row))
b0 = np.array([[1.], [0.1]])
q0 = ft.entropy_pooling(p, A0, b0)

ft.simulation_moments(R_df, q0)
```

Step 2: Incorporating Volatility Views (C1 Class)

```
# C1 views
G = np.vstack((vol_rows[-1, :], skew_row, -kurt_row))
h = np.array([[0.2**2], [-0.75], [-3.5]])
means0 = q0.T @ R

G1 = np.vstack((np.ones((1, S)),vol_rows[-1, :]))
h1 = np.array([[1.],[0.2**2]])
q1 = ft.entropy_pooling(q0, A0, b0,G1,h1)  # H2: Use q0 instead of p

ft.simulation_moments(R_df, q1)
```

Step 3: Incorporating Skewness Views (C2 Class)

```
# C2 views
G = np.vstack((vol_rows[-1, :], skew_row, -kurt_row))
h = np.array([[0.2**2], [-0.75], [-3.5]])
means0 = q1.T @ R  # Use updated parameters from q1

G2 = np.vstack((np.ones((1, S)),vol_rows[-1, :],skew_row))
h2 = np.array([[1.],[0.2**2],[-0.75]])
q2 = ft.entropy_pooling(q1, A0, b0,G2,h2)  # H2: Use q1 instead of p
ft.simulation_moments(R_df, q2)
```

Step 4: Incorporating Kurtosis Views (C3 Class)

This is the **final step** in the Sequential Entropy Pooling (H2 heuristic), where we now incorporate a **kurtosis view.**

```
# C3 views
G = np.vstack((vol_rows[-1, :], skew_row, -kurt_row))
h = np.array([[0.2**2], [-0.75], [-3.5]])
means0 = q2.T @ R  # Use updated parameters from q2

G3 = np.vstack((np.ones((1, S)),vol_rows[-1, :],skew_row,-kurt_row))
h3 = np.array([[1.],[0.2**2],[-0.75],[-3.5]])
q3 = ft.entropy_pooling(q2, A0, b0,G3,h3)  # H2: Use q2 instead of p
ft.simulation_moments(R_df, q3)

relative_entropy2 = q3.T @ (np.log(q3) - np.log(p))  # Still calculate RE against original prior
effective_number_scenarios2 = np.exp(-relative_entropy2)
print(f'RE = {np.round(relative_entropy2[0, 0], 3)}.')
print(f'Relative ENS = {np.round(effective_number_scenarios2[0, 0], 3)}.')
```

```
RE = 4.391.
Relative ENS = 0.012.
```